

Kiste v0.9.2 Engineering Drift Fix Plan

Standard config, CLI, Python API, YAML contract, and kiste-py alignment

Item	Value
Source of truth	KisteBox/kiste_spec
Implementation repo reviewed	KisteBox/kiste-py
Target release	0.9.2
Primary command	kiste doctor <path>
Canonical output	.kiste/doctor/doctor-report.json
Document purpose	Explain remaining drift, preserve good kiste-py docs, and define exact engineering fixes.

1. Executive summary

The kiste-py repository has moved significantly toward the v0.9.2 standard. The package version is now 0.9.2, the console script now points to `kiste_cli.main:main`, and `kiste_core.doctor` now contains a real v0.9.2-style doctor engine with profiles, scopes, groups, reports, and lint/security/performance checks.

The remaining work is no longer a rewrite. It is a contract cleanup pass. The most important problems are schema drift, custom-script safety, public Python API mismatch, CLI flag incompleteness, and a deeper architecture boundary issue: `kiste_cli` currently imports many modules from the public `kiste` package, which violates the split-ready rule that `kiste-cli` and `kiste-py` should be sibling wrappers over `kiste-core`.

Area	Current state	Decision
Package version	0.9.2	Keep.
Config schema	Still <code>kiste.config/v0.9.1</code>	Fix to <code>kiste.config/v0.9.2</code> .
Doctor engine	Mostly v0.9.2-like	Keep and harden.
Doctor output	Writes canonical path plus legacy path	Keep canonical, deprecate legacy.
Custom scripts	Release profile runs scripts automatically	Require <code>--execute-custom</code> .
Python API	Has write flag, lacks <code>execute_custom</code>	Expose <code>execute_custom</code> . Keep write internal.
CLI boundary	<code>kiste_cli</code> imports <code>kiste.*</code> modules	Move logic to <code>kiste_core</code> or plugin packages; CLI should not depend on <code>kiste-py</code> .

2. Source of truth rule

From v0.9.2 onward, `kiste_spec` is the canonical source of truth. Implementation repositories should not independently invent command flags, config sections, output paths, or public Python APIs. The spec repo defines the contract first; implementation repos follow it.

One quality command: `kiste doctor`

Two target scopes: `repo` / `workspace`

Four profiles: `safe` / `default` / `strict` / `release`

Stable groups: `config` / `repo` / `workspace` / `lint` / `security` / `performance` / `integration` / `custom` / `release`

One report model: `DoctorReport`

One config file: `kiste.yaml`

One lock file: `kiste.lock`

One output root: `.kiste/`

Do not add separate top-level commands for `kiste test`, `kiste lint`, `kiste security`, or `kiste performance` in v0.9.2. Also do not add `kiste doctor --scope self`. Kiste own source code is just a repo with Kiste metadata.

3. Good documentation and product language to preserve from kiste-py

The kiste-py README already has several useful documentation blocks. They should not be thrown away. They should be updated to the v0.9.2 command standard and moved into a cleaner docs structure.

Good existing doc area	Why it is useful	How to update it
Install and uv workflow	Simple onboarding for developers.	Keep uv examples; update smoke command to kiste doctor . --profile default.
Quick Start	Shows local scan, remote scan, token, workspace, data, image, GitOps, and release flows.	Replace kiste check with kiste doctor. Add doctor profiles.
Design Docs index	Points users to Getting Started, Technical Design, and SDK Design.	Keep but add Phase 9.2 Doctor Contract link from kiste_spec.
Python API examples	Shows kiste as importable library, not only CLI.	Add from kiste import doctor examples.
Generated Files section	Explains .kiste outputs.	Add .kiste/doctor/doctor-report.json and group reports.
Security Notes	Already explains token safety, GitOps secret files, image tags, datasets, and cloud safety.	Add doctor security-readiness language and avoid claiming full audit.

4. Current kiste-py alignment snapshot

Check	Observed	Status
pyproject version	version = 0.9.2	Aligned
console script	kiste = kiste_cli.main:main	Aligned direction
doctor schema	kiste.doctor/v0.9.2	Aligned
doctor profiles	safe/default/strict/release exist	Mostly aligned
doctor scopes	repo/workspace exist	Aligned
doctor groups	full group list exists	Aligned/large
doctor report path	.kiste/doctor/doctor-report.json exists	Aligned
legacy doctor path	.kiste/doctor-report.json still written	Accept temporarily
config schema	CONFIG_SCHEMA = kiste.config/v0.9.1	Drift
custom script safety	release profile executes scripts	Drift
CLI execute-custom flag	missing	Drift
Python API execute_custom	missing	Drift
CLI architecture boundary	kiste_cli imports kiste.* modules	Major drift

5. Faults to fix

Fault 1 - config schema still v0.9.1

KISTE_VERSION is 0.9.2 but CONFIG_SCHEMA and schema \$id still point to v0.9.1. This means the package says 0.9.2 while the config contract says 0.9.1.

Fault 2 - missing top-level config sections

ALLOWED_TOP_LEVEL includes doctor but still needs architecture, integrations, and outputs for the v0.9.2 standard YAML.

Fault 3 - custom scripts run too easily

custom_checks currently runs scripts in release profile. The standard requires explicit --execute-custom because custom scripts execute arbitrary repo commands.

Fault 4 - CLI lacks --execute-custom

The doctor parser has path/scope/profile/group/json, but no execute-custom flag.

Fault 5 - Python API lacks `execute_custom`

The public `kiste.doctor.run` wrapper exposes `write=True` but not `execute_custom`. Public API should expose safety control, not implementation output control.

Fault 6 - strict profile may be too shallow

`strict` currently excludes integration. The spec decision prefers `strict` as CI-quality validation; include integration if it remains `detect/plan/validate` only.

Fault 7 - migrate message still says v0.9.1

`cmd_config_migrate` prints `kiste.config/v0.9.1`. This will confuse users after upgrading to 0.9.2.

Fault 8 - CLI imports public kiste modules

`kiste_cli.main` imports from `kiste.argocd`, `kiste.blob`, `kiste.data`, and many other public SDK modules. This violates the desired dependency graph: `kiste-core -> kiste-py` and `kiste-core -> kiste-cli`.

Fault 9 - performance probes check is inaccurate

`performance.probes.readiness` currently reuses `limits_missing`. Readiness/liveness probe detection should be separate from resource-limit detection.

Fault 10 - report summary omits skip/info

summary currently returns `pass/warn/fail` only. The report contract includes `pass`, `warn`, `fail`, `skip`, and `info`.

6. Standard CLI contract to implement

```
kiste doctor <path>
kiste doctor <path> --scope repo
kiste doctor <path> --scope workspace
kiste doctor <path> --profile safe
kiste doctor <path> --profile default
kiste doctor <path> --profile strict
kiste doctor <path> --profile release
kiste doctor <path> --group lint
kiste doctor <path> --group security
kiste doctor <path> --group performance
kiste doctor <path> --group integration
kiste doctor <path> --group custom
kiste doctor <path> --execute-custom
kiste doctor <path> --json
```

Deprecated compatibility alias:

```
kiste check --repo .
# equivalent to:
kiste doctor .
# Do not document kiste check as the primary command.
```

7. Standard Python API to implement

```
from kiste import doctor

report = doctor.run(".")
report = doctor.run(".", profile="strict")
report = doctor.run(".", scope="workspace", profile="strict")
report = doctor.run(".", groups=["security", "performance"])
```

```
report = doctor.run(".", profile="release", execute_custom=True)
```

Required public signature:

```
def run(
    path: str | Path = ".",
    *,
    scope: str | None = None,
    profile: str = "default",
    groups: list[str] | None = None,
    execute_custom: bool = False,
) -> DoctorReport | dict[str, Any]:
    ...
```

The core function may keep `write=True` as an internal option, but the public wrapper should not lead with `write`. The user-facing safety option is `execute_custom`.

8. Standard YAML to implement

```
kiste: "0.9.2"
schema: "kiste.config/v0.9.2"
kind: Project
```

```
project:
  name: kiste
  type: platform
  repository: github
  description: Repo-first platform engineering toolkit
  languages:
    - python
  runtimes:
    - cli
    - library
```

```
architecture:
  mode: split-ready
  packages:
    core: packages/kiste-core
    python_sdk: packages/kiste-py
    cli: apps/kiste-cli
```

```
doctor:
  default_scope: repo
  profiles:
    safe:
      groups: [config, repo, security, blob]
    default:
      groups: [config, repo, lint, security, performance]
    strict:
      groups: [config, repo, workspace, lint, security, performance, integration]
    release:
      groups: [config, repo, architecture, module, plugin, integration, lint, security, performance,
custom, release]
  custom:
    enabled: true
    run_by_default: false
    scripts:
      - name: unit-tests
        command: pytest
```

```

    args: [tests]
    timeout_seconds: 180
    required: true
    working_dir: .

integrations:
  pulumi:
    enabled: true
    mode: detect-only
  ansible:
    enabled: true
    mode: detect-only
  cncf:
    enabled: true
    mode: detect-only
  pytorch:
    enabled: true
    mode: inspect-only

outputs:
  directory: .kiste
  doctor:
    directory: .kiste/doctor
    report: .kiste/doctor/doctor-report.json

```

9. Code patch guidance

9.1 Update config contract

```

# src/kiste_core/config.py
KISTE_VERSION = "0.9.2"
CONFIG_SCHEMA = "kiste.config/v0.9.2"

ALLOWED_TOP_LEVEL = {
    "kiste", "schema", "kind", "project", "modules",
    "architecture", "doctor", "integrations", "outputs",
    "workspace", "runtime", "services", "container", "dependencies",
    "env", "secrets", "tokens", "images", "data", "blob",
    "gitops", "argocd", "cloud", "target", "bootstrap", "release",
}

CONFIG_SCHEMA_JSON["$id"] = "https://kiste.dev/schemas/kiste-config-v0.9.2.schema.json"

```

9.2 Add execute_custom to doctor core

```

def run_doctor(
    repo_root: Path | str = ".",
    *,
    scope: str | None = None,
    profile: str = "default",
    groups: list[str] | None = None,
    execute_custom: bool = False,
    write: bool = True,
) -> dict[str, Any]:
    ...
    for group in selected_groups:
        checks.extend(run_group(group, root, selected_scope, selected_profile, execute_custom))

def custom_checks(root: Path, profile: str, execute_custom: bool) -> list[dict[str, Any]]:

```

```

scripts = configured_custom_scripts(root)
if not scripts:
    return [check("custom.configured", "custom", "warn", "No doctor custom scripts configured")]
if not execute_custom:
    return [check("custom.skipped.execution_required", "custom", "warn", "Custom scripts
configured but not executed; pass --execute-custom")]
return [run_custom_script(root, script) for script in scripts]

```

9.3 Update CLI parser and handler

```

doctor.add_argument("path", nargs="?", default=".")
doctor.add_argument("--scope", choices=("repo", "workspace"))
doctor.add_argument("--profile", choices=("safe", "default", "strict", "release"),
default="default")
doctor.add_argument("--group", action="append", dest="groups", choices=sorted(GROUPS))
doctor.add_argument("--execute-custom", action="store_true")
doctor.add_argument("--json", action="store_true")

```

```

def cmd_doctor(args: argparse.Namespace) -> int:
    report = run_doctor(
        Path(args.path),
        scope=args.scope,
        profile=args.profile,
        groups=args.groups,
        execute_custom=args.execute_custom,
    )
    if args.json:
        print(json.dumps(report, indent=2))
    else:
        print(render_doctor_report(report))
    return 1 if report["status"] == "fail" else 0

```

9.4 Update public Python wrapper

```

# src/kiste/doctor.py
from pathlib import Path
from typing import Any
from kiste_core.doctor import run_doctor

```

```

def run(
    path: str | Path = ".",
    *,
    scope: str | None = None,
    profile: str = "default",
    groups: list[str] | None = None,
    execute_custom: bool = False,
) -> dict[str, Any]:
    return run_doctor(
        path,
        scope=scope,
        profile=profile,
        groups=groups,
        execute_custom=execute_custom,
    )

```

9.5 Fix split-ready architecture boundary

Current `kiste_cli.main` imports many modules from the public `kiste` package. That makes CLI depend on `kiste-py`. The standard architecture requires both `kiste-cli` and `kiste-py` to depend on `kiste-core` instead.

```
# Target dependency rule
kiste-core    = logic, models, scanners, planners, doctor engine
kiste-py      = public Python wrappers over kiste-core
kiste-cli     = CLI parser/rendering over kiste-core

# Forbidden
kiste_cli -> kiste.* public SDK modules

# Allowed
kiste_cli -> kiste_core.*
kiste.doctor -> kiste_core.doctor
```

Practical migration path:

Step	Action
1	Leave existing kiste.* modules temporarily for compatibility.
2	Move shared logic from kiste.*_core modules into kiste_core submodules gradually.
3	Change kiste_cli imports to kiste_core imports as each module moves.
4	Keep kiste-py public modules as thin wrappers over kiste_core.
5	Add architecture tests preventing kiste_cli from importing kiste.*.

10. Acceptance criteria

Command/check	Expected result
kiste config schema	Shows kiste.config/v0.9.2 and v0.9.2 schema id.
kiste config migrate	Prints migrated to kiste.config/v0.9.2.
kiste doctor .	Writes .kiste/doctor/doctor-report.json.
kiste doctor . --profile strict --json	JSON report with schema kiste.doctor/v0.9.2.
kiste doctor . --profile release	Does not execute custom scripts automatically.
kiste doctor . --profile release --execute-custom	Runs configured custom scripts with timeout and redacted env.
from kiste import doctor; doctor.run(".")	Returns DoctorReport-compatible dict/object.
doctor.run(".", profile="release", execute_custom=True)	Runs custom only when requested.
architecture test	Fails if kiste_cli imports from public kiste package.

11. Recommended tests

```
tests/doctor/
  test_config_schema_v092.py
  test_doctor_report_path.py
  test_doctor_profiles.py
  test_doctor_groups.py
  test_doctor_execute_custom_required.py
  test_doctor_python_api.py
  test_doctor_cli_json.py
  test_doctor_legacy_report_deprecated.py

tests/architecture/
  test_cli_does_not_import_kiste_public_sdk.py
  test_py_does_not_import_cli.py
  test_core_does_not_import_py_or_cli.py
```

12. Updated README guidance to preserve

Keep the current README structure, but update the doctor-related parts as follows.

```
# Quick Start
kiste init --name robot-api
kiste scan .
kiste doctor .
kiste doctor . --profile strict

# Release check
kiste doctor . --profile release --execute-custom

# Python API
from kiste import doctor
report = doctor.run(".", profile="strict")
```

Keep these good existing documentation themes: install with uv, local and remote repo scanning, workspace graphs, data/blob safety, GitOps/SOPS safety, generated files, and security notes. The updates should route users to kiste doctor instead of kiste check or a separate kiste test command.

13. Final engineering decision

The v0.9.2 drift fix should not add many new product features. It should make the already updated kiste-py implementation obey the spec exactly. The two most urgent fixes are the schema mismatch and custom-script safety. The largest architectural follow-up is removing CLI dependence on public kiste-py modules.

Final standard:

```
kiste doctor <path>
kiste doctor <path> --profile strict
kiste doctor <path> --profile release --execute-custom

from kiste import doctor
report = doctor.run(".", profile="release", execute_custom=True)

kiste.yaml uses:
  kiste: "0.9.2"
  schema: "kiste.config/v0.9.2"

Doctor output:
  .kiste/doctor/doctor-report.json
```

Appendix A - Evidence notes

The following source observations were used to prepare this document. Line references refer to repository file reads performed during review.

Source	Observation
kiste-py pyproject.toml lines 7-10, 39-40	Version is 0.9.2 and console script points to kiste_cli.main:main.
kiste-py src/kiste_core/config.py lines 13-15, 57-60	KISTE_VERSION is 0.9.2 but CONFIG_SCHEMA and schema id still use v0.9.1.
kiste-py src/kiste_core/doctor.py lines 19-48, 60-79	Doctor schema, profiles, scopes, groups, and canonical report path exist.
kiste-py src/kiste_core/doctor.py lines 225-242	Custom script execution exists and needs explicit execute_custom gating.
kiste-py src/kiste_cli/main.py lines 131-137	Doctor CLI has path/scope/profile/group/json but not

	execute-custom.
kiste-py src/kiste_cli/main.py lines 18-76	CLI imports public kiste.* modules, creating split-ready architecture drift.
kiste-py src/kiste/doctor.py lines 13-16	Python doctor wrapper exists but lacks execute_custom.
kiste_spec Phase 9.2 contract lines 12-27, 29-39	Spec repo is source of truth and standardizes doctor as the single quality command.